Lekcja 5: Sterowanie przepływem i formatowanie w Pythonie

`#lekcja` `#python` `#podstawy` `#warunki` `#pętle`

Na tej lekcji przejdziemy do sedna programowania: nauczymy się, jak podejmować decyzje i powtarzać zadania. Opanujemy instrukcje warunkowe, które pozwalają programowi reagować na różne sytuacje, oraz pętle, które automatyzują powtarzalne czynności. Zgłębimy też zaawansowane techniki formatowania tekstu.

1. Formy operatorów przypisania

Oprócz standardowego przypisania za pomocą `=`, Python oferuje skrócone operatory, które łączą operację arytmetyczną z przypisaniem. Używanie ich sprawia, że kod jest krótszy i bardziej czytelny.

Operator	Przykład	Równoważne z	Opis
`=`	`x = 10`	-	Zwykłe przypisanie wartości.
`+=`	`x += 5`	`x = x + 5`	Dodaj i przypisz (zwiększ `x` o 5).
`-=`	`x -= 2`	`x = x - 2`	Odejmij i przypisz.
`*=`	`x *= 3`	`x = x * 3`	Pomnóż i przypisz.
`/=`	`x /= 4`	`x = x / 4`	Podziel i przypisz.
`//=`	`x //= 4`	`x = x // 4`	Wykonaj dzielenie całkowite i przypisz.
`%=`	`x %= 3`	`x = x % 3`	Oblicz resztę z dzielenia i przypisz.
`**=`	`x **= 2`	`x = x ** 2`	Podnieś do potęgi i przypisz.

2. Zaawansowane formatowanie łańcuchów znaków

Metoda `.format()`

Starsza, ale wciąż bardzo potężna metoda formatowania. Jej główną zaletą jest elastyczność i możliwość precyzyjnego kontrolowania wyglądu danych.

```
# Użycie pozycyjne
print("Imię: {0}, Wiek: {1}".format("Anna", 30))

# Użycie nazwanych argumentów (bardziej czytelne)
```

```
print("Imię: {imie}, Wiek: {wiek}".format(imie="Anna", wiek=30))

# Formatowanie liczb
cena = 4.5
print("Cena produktu: {:.2f} PLN".format(cena)) # :.2f -> zaokrąglij do 2
miejsc po przecinku
```

f-string (formatowanie literalne)

Wprowadzone w Pythonie 3.6, f-stringi są obecnie preferowanym sposobem formatowania. Są szybsze, bardziej czytelne i pozwalają umieszczać wyrażenia bezpośrednio wewnątrz nawiasów klamrowych.

> **Info**
>
> f-string to tzw. "cukier składniowy" (syntactic sugar) – ułatwienie w składni, które czyni kod przyjemniejszym w pisaniu, nie dodając nowej funkcjonalności.

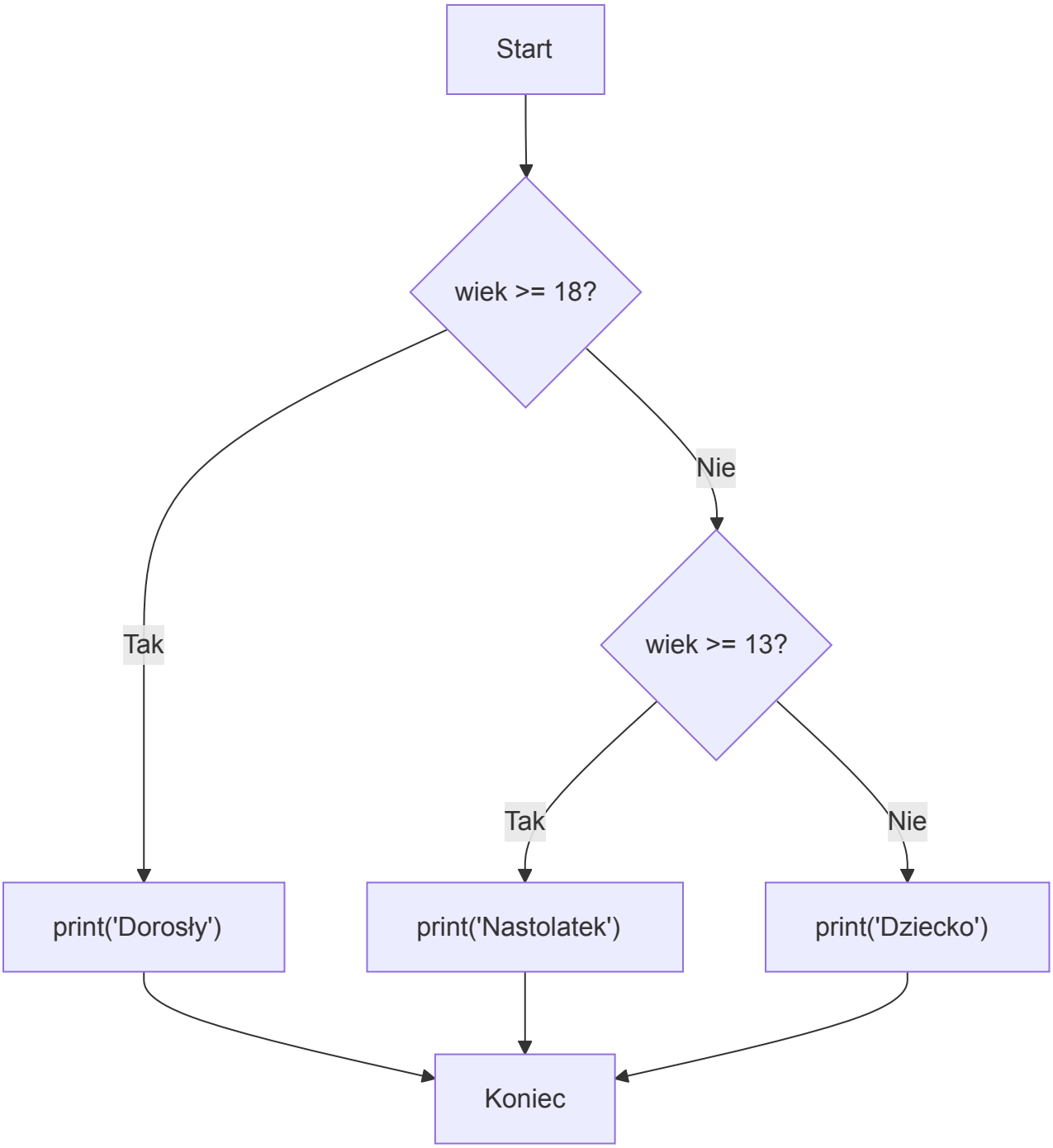
```
imie = "Anna"
wiek = 30
# Proste wstawienie zmiennych
print(f"Imię: {imie}, Wiek: {wiek}")

# Wyrażenia wewnątrz f-stringa
print(f"Za rok będziesz mieć {wiek + 1} lat.")
print(f"Twoje imię pisane wielkimi literami: {imie.upper()}")
```

3. Instrukcje warunkowe

Instrukcje warunkowe pozwalają programowi podejmować decyzje i wykonywać różne fragmenty kodu w zależności od tego, czy dany warunek jest prawdziwy (`True`), czy fałszywy (`False`).

Konstrukcja `if`, `elif`, `else`

- `if` : Sprawdza pierwszy warunek. Jeśli jest prawdziwy, wykonuje blok kodu i kończy.
- `elif` (skrót od *else if*): Jeśli poprzedni `if` był fałszywy, sprawdza kolejny warunek. Może być wiele `elif` .
- `else` : Wykonuje się, jeśli żaden z powyższych warunków (`if` i `elif`) nie był prawdziwy.




```
wiek = 15
if wiek >= 18:
    print("Jesteś osobą dorosłą.")
elif wiek >= 13:
    print("Jesteś nastolatkiem.")
else:
    print("Jesteś dzieckiem.")
```

Operatory logiczne i tożsamości

Operator	Opis	Przykład
`and`	**Koniunkcja (I)** - oba warunki muszą być prawdziwe.	`if wiek > 18 and ma_prawo_jazdy:`
`or`	**Alternatywa (LUB)** - przynajmniej jeden warunek musi być prawdziwy.	`if jest_weekend or ma_urlop:`
`not`	**Negacja (NIE)** - odwraca wartość logiczną warunku.	`if not jest_zalogowany:`
`in`	**Przynależność** - sprawdza, czy element znajduje się w sekwencji.	`if "a" in "Warszawa":`
`is`	**Tożsamość** - sprawdza, czy dwie zmienne wskazują na **ten sam obiekt** w pamięci.	`a = b = []; if a is b:`

Wyrażenie warunkowe (operator trójargumentowy)

To skondensowana forma `if-else`, przydatna do przypisywania wartości do zmiennej w jednej linii.

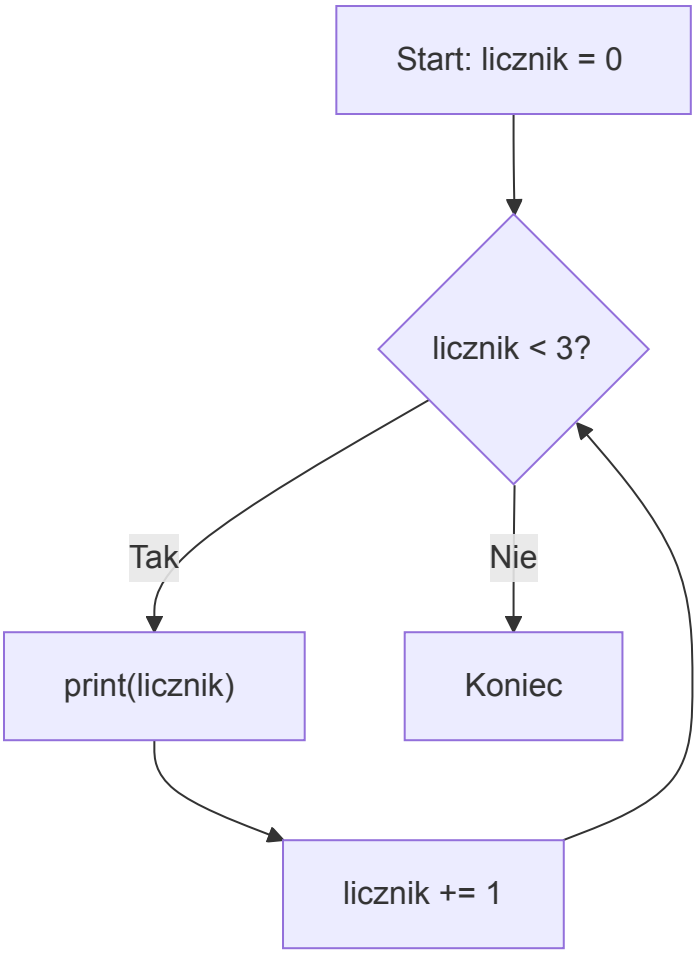
```
# Składnia: [wartość_dla_prawdy] if [warunek] else [wartość_dla_fałszu]
wiek = 20
status = "Dorosły" if wiek >= 18 else "Niepełnoletni"
print(status) # "Dorosły"
```

4. Pętle – automatyzacja powtórzeń

Pętle służą do wielokrotnego wykonywania tego samego bloku kodu.

Pętla `while`

Wykonuje blok kodu tak długo, jak jej warunek jest prawdziwy. Musisz pamiętać o zmianie wartości, od której zależy warunek, aby uniknąć pętli nieskończonej.




```
licznik = 0
while licznik < 3:
    print(f"Pętla while, iteracja: {licznik}")
    licznik += 1 # Kluczowy krok - modyfikacja licznika
```

Pętla `for`

Idealna do iterowania (przechodzenia) po elementach dowolnej sekwencji (np. listy, krotki, stringi, słowniki).

```
# Iteracja po liście
owoce = ["jabłko", "banan", "wiśnia"]
for owoc in owoce:
    print(f"Lubię owoc: {owoc}")

# Iteracja po stringu
for litera in "Python":
    print(litera, end="-") # P-y-t-h-o-n-

# Użycie funkcji range() do generowania sekwencji liczb
# range(start, stop, krok) -> generuje liczby od `start` do `stop-1`
```

```
for i in range(1, 6, 2): # Liczby od 1 do 5, z krokiem 2
    print(i) # 1, 3, 5
```

Instrukcje sterujące w pętlach: `break`, `continue`, `else`

- `break`: Natychmiast **przerywa działanie pętli** i wychodzi z niej.
- `continue`: **Przerywa bieżącą iterację** i przechodzi do następnej.
- `else`: Blok kodu, który wykonuje się, gdy pętla zakończy wszystkie swoje iteracje w sposób **naturalny** (czyli nie została przerwana przez `break`).

```
# Przykład z break i continue
print("\nPrzykład z break i continue:")
for i in range(10):
    if i == 7:
        print("Znaleziono 7, przerywam pętlę!")
        break # Zakończ pętlę
    if i % 2 == 0:
        continue # Pomiń parzyste liczby
    print(f"Przetwarzam liczbę nieparzystą: {i}")

# Przykład z else
print("\nPrzykład z else:")
szukana_liczba = 5
for i in range(1, 4):
    print(f"Sprawdzam {i}...")
    if i == szukana_liczba:
        print("Znaleziono!")
        break
else:
    # Ten blok wykona się tylko, jeśli pętla przejdzie przez wszystkie
liczby i nie znajdzie szukanej
    print("Nie znaleziono szukanej liczby w pętli.")
```

🧪 5. Zadania praktyczne

1. **Analiza wieku:** Napisz program, który pobiera od użytkownika wiek. Używając instrukcji `if-elif-else`, wyświetl jeden z komunikatów: "Niemowlę" (0-1), "Dziecko" (2-12), "Nastolatek" (13-17), "Dorosły" (18-64), "Senior" (65+).
2. **Kalkulator zniżek:** Napisz program, który oblicza cenę biletu. Cena bazowa to 100 PLN. Jeśli użytkownik jest studentem (`tak/nie`) LUB ma mniej niż 18 lat, przysługuje mu 50% zniżki. Użyj operatorów `or` i `and`.
3. **Odliczanie do startu:** Użyj pętli `while`, aby stworzyć program odliczający od 10 do 1. Po odliczeniu, poza pętlą, program powinien wyświetlić "Start!".

4. **Literowanie słowa:** Poproś użytkownika o podanie słowa. Użyj pętli `for`, aby wyświetlić każdą literę tego słowa w osobnej linii, poprzedzoną jej indeksem. Przykład dla "Kot": `0: K`, `1: o`, `2: t`.
5. **Tabela mnożenia:** Napisz program, który używając zagnieżdżonych pętli `for` (jedna w drugiej), wyświetli tabliczkę mnożenia od 1 do 5. (Wskazówka: `for i in range(1, 6): for j in range(1, 6): ...`).
6. **Gra "Zgadnij liczbę":**
   - Program "myśli" o liczbie (np. `sekret = 42`).
   - Użyj pętli `while True`, aby w nieskończoność prosić użytkownika o podanie liczby.
   - Wewnątrz pętli, sprawdź, czy podana liczba jest równa sekretnej. Jeśli tak, wyświetl gratulacje i użyj `break`, aby zakończyć grę. Jeśli nie, poinformuj, że to zła liczba.
7. **Tylko samogłoski:** Poproś użytkownika o zdanie. Użyj pętli `for` oraz instrukcji `continue`, aby wyświetlić tylko samogłoski z tego zdania. (Wskazówka: `if litera not in "aeiouy": continue`).
8. **Wyszukiwarka w liście:** Stwórz listę imion: `imiona = ["Anna", "Jan", "Piotr", "Kasia"]`. Poproś użytkownika o podanie imienia do wyszukania. Użyj pętli `for` z instrukcją `break` oraz blokiem `else`, aby:
   - Jeśli imię zostanie znalezione, wyświetlić "Znaleziono!" i przerwać pętlę.
   - Jeśli pętla zakończy się bez znalezienia imienia, wyświetlić "Nie znaleziono imienia na liście."
9. **Potęgi dwójki:** Użyj pętli `while` i skróconego operatora przypisania `*=`, aby wyświetlić potęgi liczby 2, które są mniejsze niż 1000 (1, 2, 4, 8, ..., 512).
10. **Mini-projekt: Prosty kalkulator walut:**
    - Zdefiniuj kursy w słowniku, np. `kursy = {"USD": 4.0, "EUR": 4.3}`.
    - W pętli `while True` zapytaj użytkownika o kwotę w PLN i walutę (USD/EUR).
    - Użyj `if-elif-else`, aby sprawdzić wybraną walutę i obliczyć wynik.
    - Sformatuj wynik do dwóch miejsc po przecinku, używając f-stringa.
    - Zapytaj użytkownika, czy chce kontynuować. Jeśli odpowie "nie", użyj `break`.